

02__primeros_pasos__hdfs

March 12, 2026

0.0.1 Ejercicios prácticos: Operaciones básicas en HDFS

Todos los ejercicios se deben ejecutar como usuario `root` en el contenedor del NameNode. Para cambiar al usuario `root`, ejecuta:

```
# No need to switch user, running as root
```

Ejercicio 1: Exploración de directorios Explora la estructura de directorios en HDFS usando los siguientes comandos:

```
# Listar el contenido del directorio home de root  
hdfs dfs -ls
```

```
# Listar el contenido del directorio raíz /  
hdfs dfs -ls /
```

```
# Listar el contenido del directorio /tmp  
hdfs dfs -ls /tmp
```

```
# Listar directorios con información detallada (tamaño, permisos, etc.)  
hdfs dfs -ls -h /user
```

Ejercicio 2: Crear directorios Crea una estructura de directorios en HDFS bajo tu directorio `home`:

```
# Crear un directorio llamado "datos"  
hdfs dfs -mkdir datos
```

```
# Crear un subdirectorio "datos/entrada" (requiere que "datos" exista)  
hdfs dfs -mkdir datos/entrada
```

```
# Crear múltiples directorios de una vez con -p (crea padres si no existen)  
hdfs dfs -mkdir -p datos/procesados/resultados
```

```
# Verificar que se han creado correctamente  
hdfs dfs -ls -R datos
```

Ejercicio 3: Crear archivos de prueba Crea algunos archivos de texto para practicar:

```

# Crear un archivo local de prueba
echo "Este es el contenido del archivo 1" > /tmp/archivo1.txt
echo "Contenido del archivo número 2" > /tmp/archivo2.txt

# Copiar archivos locales a HDFS
hdfs dfs -put /tmp/archivo1.txt datos/entrada/
hdfs dfs -put /tmp/archivo2.txt datos/entrada/

# Crear un archivo directamente en HDFS usando echo (a través de una tubería)
echo "Archivo creado directamente en HDFS" | hdfs dfs -put - datos/archivo_directo.txt

# Verificar los archivos creados
hdfs dfs -ls datos/entrada/
hdfs dfs -ls datos/

```

Ejercicio 4: Ver el contenido de archivos Lee el contenido de los archivos que has creado:

```

# Ver el contenido de un archivo (similar a cat en el sistema local)
hdfs dfs -cat datos/entrada/archivo1.txt

# Ver el contenido de múltiples archivos
hdfs dfs -cat datos/entrada/archivo1.txt datos/entrada/archivo2.txt

# Ver el contenido con más detalle (muestra también el número de líneas)
hdfs dfs -cat datos/archivo_directo.txt | wc -l

# Ver solo las primeras líneas de un archivo
hdfs dfs -cat datos/entrada/archivo1.txt | head -n 5

```

Ejercicio 5: Copiar y mover archivos Practica copiando y moviendo archivos dentro de HDFS:

```

# Copiar un archivo a otra ubicación
hdfs dfs -cp datos/entrada/archivo1.txt datos/copia_archivo1.txt

# Mover un archivo a otro directorio
hdfs dfs -mv datos/copia_archivo1.txt datos/procesados/

# Verificar que el archivo se ha movido
hdfs dfs -ls datos/procesados/
hdfs dfs -ls datos/entrada/

```

Ejercicio 6: Información sobre archivos y directorios Obtén información detallada sobre archivos y directorios:

```

# Ver el tamaño total de un directorio
hdfs dfs -du -h datos/

# Ver el tamaño de cada archivo dentro de un directorio

```

```
hdfs dfs -du -h -v datos/entrada/
```

```
# Contar el número de archivos en un directorio
```

```
hdfs dfs -ls -R datos/ | grep "^-" | wc -l
```

```
# Ver información detallada de un archivo específico
```

```
hdfs dfs -ls -l datos/entrada/archivo1.txt
```

```
# Ver el espacio total usado y disponible en HDFS
```

```
hdfs dfsadmin -report | grep "Configured Capacity\|DFS Used\|DFS Remaining"
```

Ejercicio 7: Cambiar permisos y propietarios Trabaja con permisos de archivos en HDFS:

```
# Cambiar permisos de un archivo (modo octal)
```

```
hdfs dfs -chmod 644 datos/entrada/archivo1.txt
```

```
# Ver los permisos actuales
```

```
hdfs dfs -ls -l datos/entrada/
```

```
# Cambiar permisos de un directorio y su contenido recursivamente
```

```
hdfs dfs -chmod -R 755 datos/
```

```
# Cambiar el propietario de un archivo (requiere permisos de admin)
```

```
# hdfs dfs -chown root:root datos/entrada/archivo1.txt
```

Ejercicio 8: Comparar archivos locales y en HDFS Compara un archivo local con uno en HDFS:

```
# Ver el contenido del archivo local
```

```
cat /tmp/archivo1.txt
```

```
# Ver el contenido del archivo en HDFS
```

```
hdfs dfs -cat datos/entrada/archivo1.txt
```

```
# Descargar un archivo de HDFS al sistema local
```

```
hdfs dfs -get datos/entrada/archivo1.txt /tmp/descargado.txt
```

```
# Comparar los dos archivos
```

```
diff /tmp/archivo1.txt /tmp/descargado.txt
```

Ejercicio 9: Eliminar archivos y directorios Practica eliminando archivos y directorios:

```
# Eliminar un archivo específico
```

```
hdfs dfs -rm datos/archivo_directo.txt
```

```
# Verificar que se ha eliminado
```

```
hdfs dfs -ls datos/
```

```
# Eliminar un directorio vacío
```

```
hdfs dfs -rmdir datos/procesados/resultados

# Eliminar un directorio con contenido (requiere -r para recursivo)
hdfs dfs -rm -r datos/procesados/

# Eliminar múltiples archivos
hdfs dfs -rm datos/entrada/archivo1.txt datos/entrada/archivo2.txt

# Verificar el estado final
hdfs dfs -ls -R datos/
```

Ejercicio 10: Limpieza final Limpia toda la estructura de directorios y archivos que creaste:

```
# Eliminar el directorio raíz con todo su contenido
hdfs dfs -rm -r datos/

# Verificar que se ha eliminado
hdfs dfs -ls

# Confirmación: el directorio home debe estar vacío (o solo contener "libros" si lo cargaste)
```

```
[1]: # Listar el contenido del directorio home de root
!hdfs dfs -ls

# Listar el contenido del directorio raíz /
!hdfs dfs -ls /

# Listar el contenido del directorio /tmp
!hdfs dfs -ls /tmp

# Listar directorios con información detallada (tamaño, permisos, etc.)
!hdfs dfs -ls -h /user
```

```
Found 1 items
drwxr-xr-x - jovyan supergroup 0 2026-03-11 17:33 libros
Found 2 items
drwxr-xr-x - jovyan supergroup 0 2026-03-11 10:47 /data
drwxr-xr-x - jovyan supergroup 0 2026-03-11 09:38 /user
ls: `/tmp': No such file or directory
Found 4 items
drwxr-xr-x - jovyan supergroup 0 2026-03-11 17:37 /user/alumnos
drwxr-xr-x - root supergroup 0 2026-03-11 09:17 /user/hadoop
drwxr-xr-x - jovyan supergroup 0 2026-03-11 17:33 /user/jovyan
drwxr-xr-x - root supergroup 0 2026-03-11 17:33 /user/root
```

```
[2]: # Crear un directorio llamado "datos"
!hdfs dfs -mkdir datos
```

```
# Crear un subdirectorio "datos/entrada" (requiere que "datos" exista)
!hdfs dfs -mkdir datos/entrada

# Crear múltiples directorios de una vez con -p (crea padres si no existen)
!hdfs dfs -mkdir -p datos/procesados/resultados

# Verificar que se han creado correctamente
!hdfs dfs -ls -R datos
```

```
drwxr-xr-x  - jovyan supergroup          0 2026-03-12 15:29 datos/entrada
drwxr-xr-x  - jovyan supergroup          0 2026-03-12 15:29 datos/procesados
drwxr-xr-x  - jovyan supergroup          0 2026-03-12 15:29
datos/procesados/resultados
```

```
[3]: # Usar en terminal - Puede dar porblemas o bloquearse en Jupyter
# Crear un archivo local de prueba
# !echo "Este es el contenido del archivo 1" > /tmp/archivo1.txt
!echo "Contenido del archivo número 2" > /tmp/archivo2.txt

# Copiar archivos locales a HDFS
# !hdfs dfs -put /tmp/archivo1.txt datos/entrada/
# !hdfs dfs -put /tmp/archivo2.txt datos/entrada/

# CREAMOS DIRECTAMENTE EN HDFS LO ANTERIOR
!echo "Este es el contenido del archivo 1" | hdfs dfs -put - datos/entrada/
↪archivo1.txt
!echo "Contenido del archivo numero 2" | hdfs dfs -put - datos/entrada/archivo2.
↪txt

# Crear un archivo directamente en HDFS usando echo (a través de una tubería)
!echo "Archivo creado directamente en HDFS" | hdfs dfs -put - datos/
↪archivo_directo.txt

# Verificar los archivos creados
!hdfs dfs -ls datos/entrada/
!hdfs dfs -ls datos/
```

```
Found 2 items
-rw-r--r--  2 jovyan supergroup          35 2026-03-12 15:29
datos/entrada/archivo1.txt
-rw-r--r--  2 jovyan supergroup          31 2026-03-12 15:29
datos/entrada/archivo2.txt
Found 3 items
-rw-r--r--  2 jovyan supergroup          36 2026-03-12 15:29
datos/archivo_directo.txt
drwxr-xr-x  - jovyan supergroup          0 2026-03-12 15:29 datos/entrada
drwxr-xr-x  - jovyan supergroup          0 2026-03-12 15:29 datos/procesados
```

```
[4]: # COMPROBACION DE CODIFICACIÓN
# import unicodedata as ud

# s = "Contenido del archivo número 2" # <- pega aquí la versión que te falla
# [(c, hex(ord(c)), ud.name(c, "?")) for c in s]
```

```
[5]: # COMPROBACION DE CODIFICACIÓN
# !locale
# !python -c "import locale, sys; print('preferred:', locale.
↳ getpreferredencoding(False)); print('fs:', sys.getfilesystemencoding())"
# !echo $LANG ; echo $LC_ALL
```

```
[6]: # Ver el contenido de un archivo (similar a cat en el sistema local)
!hdfs dfs -cat datos/entrada/archivo1.txt

# Ver el contenido de múltiples archivos
!hdfs dfs -cat datos/entrada/archivo1.txt datos/entrada/archivo2.txt

# Ver el contenido con más detalle (muestra también el número de líneas)
!hdfs dfs -cat datos/archivo_directo.txt | wc -l

# Ver solo las primeras líneas de un archivo
!hdfs dfs -cat datos/entrada/archivo1.txt | head -n 5
```

Este es el contenido del archivo 1
 Este es el contenido del archivo 1
 Contenido del archivo numero 2
 1
 Este es el contenido del archivo 1

```
[7]: # Copiar un archivo a otra ubicación
!hdfs dfs -cp datos/entrada/archivo1.txt datos/copia_archivo1.txt

# Mover un archivo a otro directorio
!hdfs dfs -mv datos/copia_archivo1.txt datos/procesados/

# Verificar que el archivo se ha movido
!hdfs dfs -ls datos/procesados/
!hdfs dfs -ls datos/entrada/
```

```
Found 2 items
-rw-r--r--    2 jovyan supergroup          35 2026-03-12 15:29
datos/procesados/copia_archivo1.txt
drwxr-xr-x    - jovyan supergroup          0 2026-03-12 15:29
datos/procesados/resultados
Found 2 items
-rw-r--r--    2 jovyan supergroup          35 2026-03-12 15:29
```

```

datos/entrada/archivo1.txt
-rw-r--r--  2 jovyan supergroup          31 2026-03-12 15:29
datos/entrada/archivo2.txt

```

```

[8]: # Ver el tamaño total de un directorio
!hdfs dfs -du -h datos/

# Ver el tamaño de cada archivo dentro de un directorio
!hdfs dfs -du -h -v datos/entrada/

# Contar el número de archivos en un directorio
!hdfs dfs -ls -R datos/ | grep "^-" | wc -l

# Ver información detallada de un archivo específico
!hdfs dfs -ls datos/entrada/archivo1.txt

# Ver el espacio total usado y disponible en HDFS
!hdfs dfsadmin -report | grep "Configured Capacity\|DFS Used\|DFS Remaining"

```

```

36  72  datos/archivo_directo.txt
66 132  datos/entrada
35  70  datos/procesados
SIZE  DISK_SPACE_CONSUMED_WITH_ALL_REPLICAS  FULL_PATH_NAME
35    70                                     datos/entrada/archivo1.txt
31    62                                     datos/entrada/archivo2.txt
4
-rw-r--r--  2 jovyan supergroup          35 2026-03-12 15:29
datos/entrada/archivo1.txt
Configured Capacity: 2162202353664 (1.97 TB)
DFS Remaining: 1886634704896 (1.72 TB)
DFS Used: 39561453930 (36.84 GB)
DFS Used%: 2.05%
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 19780755637 (18.42 GB)
Non DFS Used: 63010713419 (58.68 GB)
DFS Remaining: 943317352448 (878.53 GB)
DFS Used%: 1.83%
DFS Remaining%: 87.26%
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 19780698293 (18.42 GB)
Non DFS Used: 63010770763 (58.68 GB)
DFS Remaining: 943317352448 (878.53 GB)
DFS Used%: 1.83%
DFS Remaining%: 87.26%

```

```

[9]: # Cambiar permisos de un archivo (modo octal)
!hdfs dfs -chmod 644 datos/entrada/archivo1.txt

```

```

# Ver los permisos actuales
!hdfs dfs -ls datos/entrada/

# Cambiar permisos de un directorio y su contenido recursivamente
!hdfs dfs -chmod -R 755 datos/

# Cambiar el propietario de un archivo (requiere permisos de admin)
# hdfs dfs -chown root:root datos/entrada/archivo1.txt

```

```

Found 2 items
-rw-r--r--    2 jovyan supergroup          35 2026-03-12 15:29
datos/entrada/archivo1.txt
-rw-r--r--    2 jovyan supergroup          31 2026-03-12 15:29
datos/entrada/archivo2.txt

```

```

[10]: # Ver el contenido del archivo local
!cat /tmp/archivo1.txt

# Ver el contenido del archivo en HDFS
!hdfs dfs -cat datos/entrada/archivo1.txt

# Descargar un archivo de HDFS al sistema local
!hdfs dfs -get datos/entrada/archivo1.txt /tmp/descargado.txt

# Comparar los dos archivos
!diff /tmp/archivo1.txt /tmp/descargado.txt

```

```

cat: /tmp/archivo1.txt: No such file or directory
Este es el contenido del archivo 1
get: `/tmp/descargado.txt': File exists
diff: /tmp/archivo1.txt: No such file or directory

```

```

[11]: # Eliminar un archivo específico
!hdfs dfs -rm datos/archivo_directo.txt

# Verificar que se ha eliminado
!hdfs dfs -ls datos/

# Eliminar un directorio vacío
!hdfs dfs -rmdir datos/procesados/resultados

# Eliminar un directorio con contenido (requiere -r para recursivo)
!hdfs dfs -rm -r datos/procesados/

# Eliminar múltiples archivos
!hdfs dfs -rm datos/entrada/archivo1.txt datos/entrada/archivo2.txt

```



```
# Verificar el estado final
!hdfs dfs -ls -R datos/
```

```
Deleted datos/archivo_directo.txt
Found 2 items
drwxr-xr-x   - jovyan supergroup          0 2026-03-12 15:29 datos/entrada
drwxr-xr-x   - jovyan supergroup          0 2026-03-12 15:29 datos/procesados
Deleted datos/procesados
Deleted datos/entrada/archivo1.txt
Deleted datos/entrada/archivo2.txt
drwxr-xr-x   - jovyan supergroup          0 2026-03-12 15:30 datos/entrada
```

```
[12]: # Eliminar el directorio raíz con todo su contenido
!hdfs dfs -rm -r datos/

# Verificar que se ha eliminado
!hdfs dfs -ls

# Confirmación: el directorio home debe estar vacío (o solo contener "libros"
↳ si lo cargaste)
```

```
Deleted datos
Found 1 items
drwxr-xr-x   - jovyan supergroup          0 2026-03-11 17:33 libros
```

0.0.2 Prueba de acceso a HDFS y carga de datos

1. En el Namenode, conviértete en usuario no privilegiado (haciendo, como root, `# No need to switch user, running as root`)
2. Comprueba que puedes acceder al HDFS ejecutando:

```
hdfs dfs -ls
```

(Este comando no debería dar ninguna salida, ya que no tenemos nada en ese directorio).

3. Descarga de <https://github.com/dsevilla/bd2-data/releases/download/libros-tar-gz/libros.tar.gz> y descomprime los ficheros de ejemplo usando:

```
wget https://github.com/dsevilla/bd2-data/releases/download/libros-tar-gz/libros.tar.gz -q0 -
| tar -xzf - -C /tmp
```

4. Siguiendo en el NameNode como usuario root, copia los datos a HDFS:

```
cd /tmp
hdfs dfs -put libros .
hdfs dfs -ls libros
```

Una vez copiados borra el directorio libros del disco local (`rm -rf /tmp/libros*`).

5. Mira en el interfaz web del HDFS dónde se encuentran los bloques correspondientes al fichero **random_words.txt.bz2**. Comprueba que cada bloque tiene 2 réplicas.

- Abre el navegador y conéctate al interfaz web del NameNode.
- Ve al menú “Utilities” -> “Browse the filesystem”.

```
[13]: !hdfs dfs -ls
```

```
Found 1 items
drwxr-xr-x   - jovyan supergroup          0 2026-03-11 17:33 libros
```

```
[14]: !wget https://github.com/dsevilla/bd2-data/releases/download/libros-tar-gz/
      ↪libros.tar.gz -q0 -N
      ! | tar -xzf - -C /tmp
```

```
libros/
libros/pg17013.txt.gz
libros/pg16625.txt.gz
libros/pg25807.txt.gz
libros/pg32315.txt.gz
libros/random_words.txt.bz2
libros/pg1619.txt.gz
libros/pg7109.txt.gz
libros/pg2000.txt.gz
libros/pg14329.txt.gz
libros/pg24536.txt.gz
libros/pg25640.txt.gz
libros/pg8870.txt.gz
libros/pg18005.txt.gz
libros/pg5201.txt.gz
libros/pg9980.txt.gz
libros/pg17073.txt.gz
```

```
[15]: %cd /tmp
      !hdfs dfs -put libros .
      !hdfs dfs -ls libros
```

```
/tmp
put: `libros/pg2000.txt.gz': File exists
put: `libros/pg1619.txt.gz': File exists
put: `libros/pg14329.txt.gz': File exists
put: `libros/pg9980.txt.gz': File exists
put: `libros/pg7109.txt.gz': File exists
put: `libros/random_words.txt.bz2': File exists
put: `libros/pg17013.txt.gz': File exists
put: `libros/pg8870.txt.gz': File exists
put: `libros/pg25640.txt.gz': File exists
put: `libros/pg16625.txt.gz': File exists
put: `libros/pg18005.txt.gz': File exists
```

```

put: `libros/pg32315.txt.gz': File exists
put: `libros/pg5201.txt.gz': File exists
put: `libros/pg24536.txt.gz': File exists
put: `libros/pg25807.txt.gz': File exists
put: `libros/pg17073.txt.gz': File exists
Found 16 items
-rw-r--r--    2 jovyan supergroup      441804 2026-03-11 17:33
libros/pg14329.txt.gz
-rw-r--r--    2 jovyan supergroup      264123 2026-03-11 17:33
libros/pg1619.txt.gz
-rw-r--r--    2 jovyan supergroup      455129 2026-03-11 17:33
libros/pg16625.txt.gz
-rw-r--r--    2 jovyan supergroup      939502 2026-03-11 17:33
libros/pg17013.txt.gz
-rw-r--r--    2 jovyan supergroup      737367 2026-03-11 17:33
libros/pg17073.txt.gz
-rw-r--r--    2 jovyan supergroup      219304 2026-03-11 17:33
libros/pg18005.txt.gz
-rw-r--r--    2 jovyan supergroup      813698 2026-03-11 17:33
libros/pg2000.txt.gz
-rw-r--r--    2 jovyan supergroup      328494 2026-03-11 17:33
libros/pg24536.txt.gz
-rw-r--r--    2 jovyan supergroup      504188 2026-03-11 17:33
libros/pg25640.txt.gz
-rw-r--r--    2 jovyan supergroup       38194 2026-03-11 17:33
libros/pg25807.txt.gz
-rw-r--r--    2 jovyan supergroup     103986 2026-03-11 17:33
libros/pg32315.txt.gz
-rw-r--r--    2 jovyan supergroup     125693 2026-03-11 17:33
libros/pg5201.txt.gz
-rw-r--r--    2 jovyan supergroup       82099 2026-03-11 17:33
libros/pg7109.txt.gz
-rw-r--r--    2 jovyan supergroup      99685 2026-03-11 17:33
libros/pg8870.txt.gz
-rw-r--r--    2 jovyan supergroup       85187 2026-03-11 17:33
libros/pg9980.txt.gz
-rw-r--r--    2 jovyan supergroup    330326458 2026-03-11 17:33
libros/random_words.txt.bz2

```

0.0.3 Acceso a HDFS desde python con pyarrow

PyArrow proporciona una interfaz Python para interactuar con HDFS a través de su módulo `pyarrow.fs.HadoopFileSystem`. Esto nos permite realizar operaciones sobre HDFS directamente desde Python sin necesidad de usar comandos de shell.

Preparación del entorno Como usuario `root` en el contenedor `namenode`, ejecuta los siguientes pasos:

1. Crear un entorno virtual Python:

```
# Conectarse al contenedor namenode
docker exec -it namenode /bin/bash

# Cambiar al usuario root
# No need to switch user, running as root

# Crear el entorno virtual
python3 -m venv ~/.venv

# Activar el entorno virtual
source ~/.venv/bin/activate
```

2. Instalar PyArrow:

```
# Con el entorno virtual activado
pip install pyarrow
```

3. Configurar las variables de entorno para Hadoop:

```
export CLASSPATH="$(hadoop classpath --glob):$CLASSPATH"
```

4. Verificar la instalación:

```
python3 -c "import pyarrow as pa; print(f'PyArrow version: {pa.__version__}')"

```

```
[16]: # Conectarse al contenedor namenode
!docker exec -it namenode /bin/bash

# Cambiar al usuario root
# No need to switch user, running as root

# Crear el entorno virtual
!python3 -m venv ~/.venv

# Activar el entorno virtual
# source ~/.venv/bin/activate # 'source' no funciona en celdas, usar variables_
↪de entorno Python
```

```
/bin/bash: line 1: docker: command not found
Error: [Errno 13] Permission denied: '/home/jovyan/.venv/pyvenv.cfg'
```

```
[17]: # Con el entorno virtual activado
!pip install pyarrow
```

```
Requirement already satisfied: pyarrow in /opt/conda/lib/python3.11/site-
packages (23.0.1)
```

```
[18]: !export CLASSPATH="$(hadoop classpath --glob):$CLASSPATH"
```

```
[19]: !python3 -c "import pyarrow as pa; print(f'PyArrow version: {pa.__version__}')
```

PyArrow version: 23.0.1

Ejemplos de uso del API de PyArrow FileSystem A continuación se muestran ejemplos equivalentes a los comandos `hdfs dfs` vistos anteriormente, pero usando Python y PyArrow:

```
[20]: %pip install -q --upgrade pyarrow
```

Note: you may need to restart the kernel to use updated packages.

```
[21]: # Execute hadoop classpath --glob and set CLASSPATH environment variable
import os
import subprocess

classpath = subprocess.check_output(["hadoop", "classpath", "--glob"]).
    ↪ decode("utf-8").strip()
os.environ["CLASSPATH"] = classpath
```

Ejemplo 1: Conectar a HDFS y listar directorios

```
[22]: import pyarrow as pa
from pyarrow import fs
from pyarrow.fs import FileInfo, FileSystem

# Conectar a HDFS (namenode en puerto 9000)
hdfs: FileSystem = fs.HadoopFileSystem(host="namenode", port=9000,
    ↪ user="jovyan")

def print_file_list_info(file_info_list: list[FileInfo]):
    for file_info in file_info_list:
        file_type: str | None = file_info.type.name if file_info.type else
            ↪ "UNKNOWN"
        file_size: int = file_info.size if file_info.size is not None else 0
        print(f"{file_type:10} {file_size:>10} {file_info.path}")

# Listar contenido del directorio home del usuario
print("=== Contenido del directorio home ===")
file_info_list: list[FileInfo] = hdfs.get_file_info(fs.FileSelector("/user/
    ↪ jovyan"))
print_file_list_info(file_info_list)

# Listar el directorio raíz
print("\n=== Contenido del directorio raíz / ===")
root_info: list[FileInfo] = hdfs.get_file_info(fs.FileSelector("/"))
```

```
print_file_list_info(root_info)
```

```
=== Contenido del directorio home ===  
Directory          0 /user/jovyan/libros
```

```
=== Contenido del directorio raíz / ===  
Directory          0 /data  
Directory          0 /user
```

Ejemplo 2: Crear directorios

```
[23]: # Crear un directorio simple  
hdfs.create_dir("/user/root/datos_python")  
print("Directorio 'datos_python' creado")  
  
# Crear estructura de directorios (equivalente a mkdir -p)  
hdfs.create_dir("/user/root/datos_python/entrada")  
hdfs.create_dir("/user/root/datos_python/procesados/resultados")  
print("Estructura de directorios creada")  
  
# Verificar creación  
file_list: list[FileInfo] = hdfs.get_file_info(  
    fs.FileSelector("/user/root/datos_python", recursive=True)  
)  
print("=== Estructura creada ===")  
print_file_list_info(file_list)  
  
Directorio 'datos_python' creado  
Estructura de directorios creada  
=== Estructura creada ===  
Directory          0 /user/root/datos_python/entrada  
Directory          0 /user/root/datos_python/procesados  
Directory          0 /user/root/datos_python/procesados/resultados
```

Ejemplo 3: Escribir archivos en HDFS

```
[24]: # Escribir un archivo de texto directamente en HDFS  
with hdfs.open_output_stream("/user/root/datos_python/entrada/archivo1.txt") as f:  
    f.write(b"Este es el contenido del archivo 1 desde Python\n")  
    f.write("Segunda línea del archivo\n".encode())  
  
print("Archivo archivo1.txt creado")  
  
# Escribir otro archivo  
contenido = "Contenido del archivo número 2 desde PyArrow"  
with hdfs.open_output_stream("/user/root/datos_python/entrada/archivo2.txt") as f:  
    f.write(contenido.encode())
```

```

        f.write(contenido.encode("utf-8"))

print("Archivo archivo2.txt creado")

# Copiar un archivo local a HDFS
import os

# Crear archivo local temporal
local_file = "/tmp/archivo_local.txt"
with open(local_file, "w") as f:
    f.write("Contenido desde archivo local\n")

# Copiar a HDFS usando copy_file
with open(local_file, "rb") as local_f:
    with hdfs.open_output_stream("/user/root/datos_python/desde_local.txt") as hdfs_f:
        hdfs_f.write(local_f.read())

print("Archivo copiado desde local a HDFS")

# Limpiar archivo local
os.remove(local_file)

```

Archivo archivo1.txt creado
 Archivo archivo2.txt creado
 Archivo copiado desde local a HDFS

Ejemplo 4: Leer archivos de HDFS

```

[25]: # Leer contenido de un archivo (equivalente a hdfs dfs -cat)
with hdfs.open_input_stream("/user/root/datos_python/entrada/archivo1.txt") as f:
    contenido: bytes = f.read()
    print("=== Contenido de archivo1.txt ===")
    print(contenido.decode("utf-8"))

# Leer con readall (más eficiente para archivos completos)
contenido2: bytes = hdfs.open_input_stream(
    "/user/root/datos_python/entrada/archivo2.txt"
).readall()
print("\n=== Contenido de archivo2.txt ===")
print(contenido2.decode("utf-8"))

```

=== Contenido de archivo1.txt ===
 Este es el contenido del archivo 1 desde Python
 Segunda línea del archivo

```
=== Contenido de archivo2.txt ===  
Contenido del archivo número 2 desde PyArrow
```

Ejemplo 5: Copiar y mover archivos dentro de HDFS

[26]: *# Copiar un archivo (PyArrow no tiene copy directo, hay que leer y escribir)*

```
def copy_file_hdfs(hdfs_fs, src, dst):  
    """Copia un archivo en HDFS de src a dst"""  
    with hdfs_fs.open_input_stream(src) as f_in:  
        with hdfs_fs.open_output_stream(dst) as f_out:  
            f_out.write(f_in.read())  
  
# Copiar archivo  
copy_file_hdfs(  
    hdfs,  
    "/user/root/datos_python/entrada/archivo1.txt",  
    "/user/root/datos_python/copia_archivo1.txt",  
)  
print("Archivo copiado")  
  
# Mover archivo (equivalente a hdfs dfs -mv)  
hdfs.move(  
    "/user/root/datos_python/copia_archivo1.txt",  
    "/user/root/datos_python/procesados/archivo_movido.txt",  
)  
print("Archivo movido")  
  
# Verificar  
print_file_list_info(hdfs.get_file_info(fs.FileSelector("/user/root/  
↪datos_python/procesados")))
```

Archivo copiado

Archivo movido

File	75 /user/root/datos_python/procesados/archivo_movido.txt
Directory	0 /user/root/datos_python/procesados/resultados

Ejemplo 6: Obtener información de archivos

[27]: `from pyarrow.fs import FileInfo`

```
# Información detallada de un archivo específico  
file_info: FileInfo = hdfs.get_file_info("/user/root/datos_python/entrada/  
↪archivo1.txt")  
print("=== Información de archivo ===")  
print(f"Ruta: {file_info.path}")  
print(f"Tipo: {file_info.type.name}")
```



```

print(f"Tamaño: {file_info.size} bytes")
print(f"Tiempo modificación: {file_info.mtime}")

# Tamaño de un directorio (recursivo)
dir_selector = fs.FileSelector("/user/root/datos_python", recursive=True)
total_size = 0
file_count = 0

for info in hdfs.get_file_info(dir_selector):
    if info.type == fs.FileType.File:
        total_size += info.size
        file_count += 1
        print(f"{info.size:>10} bytes {info.path}")

print(f"\nTotal: {file_count} archivos, {total_size} bytes")

```

```

=== Información de archivo ===
Ruta: /user/root/datos_python/entrada/archivo1.txt
Tipo: File
Tamaño: 75 bytes
Tiempo modificación: 2026-03-12 15:31:48+00:00
    30 bytes /user/root/datos_python/desde_local.txt
    75 bytes /user/root/datos_python/entrada/archivo1.txt
    45 bytes /user/root/datos_python/entrada/archivo2.txt
    75 bytes /user/root/datos_python/procesados/archivo_movido.txt

Total: 4 archivos, 225 bytes

```

Ejemplo 7: Eliminar archivos y directorios

```

[28]: # Eliminar un archivo específico
hdfs.delete_file("/user/root/datos_python/desde_local.txt")
print("Archivo eliminado")

# Eliminar directorio con contenido (equivalente a hdfs dfs -rm -r)
hdfs.delete_dir("/user/root/datos_python/procesados")
print("Directorio 'procesados' eliminado recursivamente")

# Verificar eliminación
remaining: list[FileInfo] = hdfs.get_file_info(
    fs.FileSelector("/user/root/datos_python", recursive=True)
)
print("\n=== Archivos restantes ===")
for info in remaining:
    print(f"{info.type.name:10} {info.path}")

```

```

Archivo eliminado
Directorio 'procesados' eliminado recursivamente

```

```

=== Archivos restantes ===
Directory /user/root/datos_python/entrada
File      /user/root/datos_python/entrada/archivo1.txt
File      /user/root/datos_python/entrada/archivo2.txt

```

Ejemplo 8: Trabajar con PyArrow Tables y Parquet PyArrow es especialmente útil para trabajar con datos en formato Parquet directamente en HDFS:

```

[29]: import pandas as pd
import pyarrow.parquet as pq

# Crear un DataFrame de ejemplo
df = pd.DataFrame(
    {
        "id": range(1, 101),
        "nombre": [f"Producto_{i}" for i in range(1, 101)],
        "precio": [10.5 + i * 0.5 for i in range(100)],
        "categoria": ["A", "B", "C", "D"] * 25,
    }
)

# Convertir a tabla de PyArrow
table: pa.Table = pa.Table.from_pandas(df)

# Escribir como Parquet en HDFS
pq.write_table(table, "/user/root/datos_python/productos.parquet",
    ↪filesystem=hdfs)
print("Archivo Parquet escrito en HDFS")

# Leer el archivo Parquet desde HDFS
tabla_leida: pa.Table = pq.read_table("/user/root/datos_python/productos.
    ↪parquet", filesystem=hdfs)
print("\n=== Tabla leída desde HDFS ===")
print(f"Filas: {tabla_leida.num_rows}, Columnas: {tabla_leida.num_columns}")
print(tabla_leida.schema)

# Convertir de vuelta a pandas
df_leido: pd.DataFrame = tabla_leida.to_pandas()
print("\nPrimeras filas:")
print(df_leido.head())

```

Archivo Parquet escrito en HDFS

```

=== Tabla leída desde HDFS ===
Filas: 100, Columnas: 4
id: int64
nombre: string

```

```

precio: double
categoria: string
-- schema metadata --
pandas: '{"index_columns": [{"kind": "range", "name": null, "start": 0, "' + 723

```

Primeras filas:

	id	nombre	precio	categoria
0	1	Producto_1	10.5	A
1	2	Producto_2	11.0	B
2	3	Producto_3	11.5	C
3	4	Producto_4	12.0	D
4	5	Producto_5	12.5	A

Ejemplo 9: Limpieza final

```

[30]: # Eliminar todo el directorio de prueba
hdfs.delete_dir("/user/root/datos_python")
print("Directorio 'datos_python' eliminado completamente")

# Verificar que el directorio home está limpio
print("=== Contenido final del directorio home ===")
print_file_list_info(hdfs.get_file_info(fs.FileSelector("/user/root")))

```

```

Directorio 'datos_python' eliminado completamente
=== Contenido final del directorio home ===
Directory          0 /user/root/libros

```

Ventajas de usar PyArrow para HDFS

- **Integración nativa:** Conexión directa con HDFS sin necesidad de comandos shell
- **Formato Parquet:** Lectura/escritura eficiente de datos columnares
- **Rendimiento:** Operaciones optimizadas en C++ subyacente
- **Portabilidad:** Mismo código funciona con diferentes sistemas de archivos (HDFS, S3, local)
- **Pythonic:** API orientada a objetos más natural para desarrolladores Python

Notas importantes

- **Conexión HDFS:** Usar `host='namenode'` y `port=9000` (configurado en `core-site.xml`). PyArrow permite también un caso especial con `hdfs://default:0`, para que el código funcione en cualquier entorno Hadoop una vez que PyArrow tiene acceso a la configuración de Hadoop.
- **Usuario:** Especificar `user='root'` para mantener permisos consistentes
- **Rutas:** Siempre usar rutas absolutas (comenzando con `/`)
- **Tipos de archivo:** PyArrow distingue entre `FileType.File`, `FileType.Directory`, y `FileType.NotFound`
- **Encoding:** Recordar codificar/decodificar strings a bytes cuando sea necesario

0.0.4 Acceso con Visual Studio Code al clúster a través de un servidor Jupyter

El contenedor del NameNode/ResourceManager se puede ejecutar un servidor Jupyter que permite acceder al clúster Hadoop a través de una URL de conexión a un servidor Jupyter, y por lo tanto, a través de Visual Studio Code (y también de la web).

Existen varias formas de hacerlo. Por ejemplo, usando la utilidad “DevContainer” de Visual Studio Code, que permite definir un entorno de desarrollo basado en contenedores Docker. Sin embargo, lo haremos manualmente.

A continuación se explica cómo hacerlo.

Lo primero, suponemos que existe un virtual environment de Python para el usuario `root`. Si no es así, créalo y actívalo:

```
python3 -m venv ~/.venv
. ~/.venv/bin/activate
```

Si ya existe, sólo hace falta activarlo:

```
. ~/.venv/bin/activate
```

```
[31]: !python3 -m venv ~/.venv
      !. ~/.venv/bin/activate
```

```
Error: [Errno 13] Permission denied: '/home/jovyan/.venv/pyvenv.cfg'
```

```
[32]: !. ~/.venv/bin/activate
```

Lo siguiente es instalar los paquetes necesarios:

```
pip install jupyter
```

```
[33]: !pip install jupyter
```

```
Requirement already satisfied: jupyter in /opt/conda/lib/python3.11/site-
packages (1.1.1)
Requirement already satisfied: notebook in /opt/conda/lib/python3.11/site-
packages (from jupyter) (7.0.6)
Requirement already satisfied: jupyter-console in
/opt/conda/lib/python3.11/site-packages (from jupyter) (6.6.3)
Requirement already satisfied: nbconvert in /opt/conda/lib/python3.11/site-
packages (from jupyter) (7.9.2)
Requirement already satisfied: ipykernel in /opt/conda/lib/python3.11/site-
packages (from jupyter) (6.25.2)
Requirement already satisfied: ipywidgets in /opt/conda/lib/python3.11/site-
packages (from jupyter) (8.1.1)
Requirement already satisfied: jupyterlab in /opt/conda/lib/python3.11/site-
packages (from jupyter) (4.0.7)
Requirement already satisfied: comm>=0.1.1 in /opt/conda/lib/python3.11/site-
packages (from ipykernel->jupyter) (0.1.4)
Requirement already satisfied: debugpy>=1.6.5 in /opt/conda/lib/python3.11/site-
```

packages (from ipykernel->jupyter) (1.8.0)
 Requirement already satisfied: ipython>=7.23.1 in
 /opt/conda/lib/python3.11/site-packages (from ipykernel->jupyter) (8.16.1)
 Requirement already satisfied: jupyter-client>=6.1.12 in
 /opt/conda/lib/python3.11/site-packages (from ipykernel->jupyter) (8.4.0)
 Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in
 /opt/conda/lib/python3.11/site-packages (from ipykernel->jupyter) (5.4.0)
 Requirement already satisfied: matplotlib-inline>=0.1 in
 /opt/conda/lib/python3.11/site-packages (from ipykernel->jupyter) (0.1.6)
 Requirement already satisfied: nest-asyncio in /opt/conda/lib/python3.11/site-
 packages (from ipykernel->jupyter) (1.5.8)
 Requirement already satisfied: packaging in /opt/conda/lib/python3.11/site-
 packages (from ipykernel->jupyter) (23.2)
 Requirement already satisfied: psutil in /opt/conda/lib/python3.11/site-packages
 (from ipykernel->jupyter) (5.9.5)
 Requirement already satisfied: pyzmq>=20 in /opt/conda/lib/python3.11/site-
 packages (from ipykernel->jupyter) (25.1.1)
 Requirement already satisfied: tornado>=6.1 in /opt/conda/lib/python3.11/site-
 packages (from ipykernel->jupyter) (6.3.3)
 Requirement already satisfied: traitlets>=5.4.0 in
 /opt/conda/lib/python3.11/site-packages (from ipykernel->jupyter) (5.11.2)
 Requirement already satisfied: widgetsnbextension~=4.0.9 in
 /opt/conda/lib/python3.11/site-packages (from ipywidgets->jupyter) (4.0.9)
 Requirement already satisfied: jupyterlab-widgets~=3.0.9 in
 /opt/conda/lib/python3.11/site-packages (from ipywidgets->jupyter) (3.0.9)
 Requirement already satisfied: prompt-toolkit>=3.0.30 in
 /opt/conda/lib/python3.11/site-packages (from jupyter-console->jupyter) (3.0.39)
 Requirement already satisfied: pygments in /opt/conda/lib/python3.11/site-
 packages (from jupyter-console->jupyter) (2.16.1)
 Requirement already satisfied: async-lru>=1.0.0 in
 /opt/conda/lib/python3.11/site-packages (from jupyterlab->jupyter) (2.0.4)
 Requirement already satisfied: jinja2>=3.0.3 in /opt/conda/lib/python3.11/site-
 packages (from jupyterlab->jupyter) (3.1.2)
 Requirement already satisfied: jupyter-lsp>=2.0.0 in
 /opt/conda/lib/python3.11/site-packages (from jupyterlab->jupyter) (2.2.0)
 Requirement already satisfied: jupyter-server<3,>=2.4.0 in
 /opt/conda/lib/python3.11/site-packages (from jupyterlab->jupyter) (2.8.0)
 Requirement already satisfied: jupyterlab-server<3,>=2.19.0 in
 /opt/conda/lib/python3.11/site-packages (from jupyterlab->jupyter) (2.25.0)
 Requirement already satisfied: notebook-shim>=0.2 in
 /opt/conda/lib/python3.11/site-packages (from jupyterlab->jupyter) (0.2.3)
 Requirement already satisfied: beautifulsoup4 in /opt/conda/lib/python3.11/site-
 packages (from nbconvert->jupyter) (4.12.2)
 Requirement already satisfied: bleach!=5.0.0 in /opt/conda/lib/python3.11/site-
 packages (from nbconvert->jupyter) (6.1.0)
 Requirement already satisfied: defusedxml in /opt/conda/lib/python3.11/site-
 packages (from nbconvert->jupyter) (0.7.1)
 Requirement already satisfied: jupyterlab-pygments in

/opt/conda/lib/python3.11/site-packages (from nbconvert->jupyter) (0.2.2)
 Requirement already satisfied: markupsafe>=2.0 in
 /opt/conda/lib/python3.11/site-packages (from nbconvert->jupyter) (2.1.3)
 Requirement already satisfied: mistune<4,>=2.0.3 in
 /opt/conda/lib/python3.11/site-packages (from nbconvert->jupyter) (3.0.1)
 Requirement already satisfied: nbclient>=0.5.0 in
 /opt/conda/lib/python3.11/site-packages (from nbconvert->jupyter) (0.8.0)
 Requirement already satisfied: nbformat>=5.7 in /opt/conda/lib/python3.11/site-
 packages (from nbconvert->jupyter) (5.9.2)
 Requirement already satisfied: pandocfilters>=1.4.1 in
 /opt/conda/lib/python3.11/site-packages (from nbconvert->jupyter) (1.5.0)
 Requirement already satisfied: tinycss2 in /opt/conda/lib/python3.11/site-
 packages (from nbconvert->jupyter) (1.2.1)
 Requirement already satisfied: six>=1.9.0 in /opt/conda/lib/python3.11/site-
 packages (from bleach!=5.0.0->nbconvert->jupyter) (1.16.0)
 Requirement already satisfied: webencodings in /opt/conda/lib/python3.11/site-
 packages (from bleach!=5.0.0->nbconvert->jupyter) (0.5.1)
 Requirement already satisfied: backcall in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (0.2.0)
 Requirement already satisfied: decorator in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (5.1.1)
 Requirement already satisfied: jedi>=0.16 in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (0.19.1)
 Requirement already satisfied: pickleshare in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (0.7.5)
 Requirement already satisfied: stack-data in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (0.6.2)
 Requirement already satisfied: pexpect>4.3 in /opt/conda/lib/python3.11/site-
 packages (from ipython>=7.23.1->ipykernel->jupyter) (4.8.0)
 Requirement already satisfied: python-dateutil>=2.8.2 in
 /opt/conda/lib/python3.11/site-packages (from jupyter-
 client>=6.1.12->ipykernel->jupyter) (2.8.2)
 Requirement already satisfied: platformdirs>=2.5 in
 /opt/conda/lib/python3.11/site-packages (from jupyter-
 core!=5.0.*,>=4.12->ipykernel->jupyter) (3.11.0)
 Requirement already satisfied: anyio>=3.1.0 in /opt/conda/lib/python3.11/site-
 packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (4.0.0)
 Requirement already satisfied: argon2-cffi in /opt/conda/lib/python3.11/site-
 packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (23.1.0)
 Requirement already satisfied: jupyter-events>=0.6.0 in
 /opt/conda/lib/python3.11/site-packages (from jupyter-
 server<3,>=2.4.0->jupyterlab->jupyter) (0.8.0)
 Requirement already satisfied: jupyter-server-terminals in
 /opt/conda/lib/python3.11/site-packages (from jupyter-
 server<3,>=2.4.0->jupyterlab->jupyter) (0.4.4)
 Requirement already satisfied: overrides in /opt/conda/lib/python3.11/site-
 packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (7.4.0)
 Requirement already satisfied: prometheus-client in

/opt/conda/lib/python3.11/site-packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (0.17.1)
 Requirement already satisfied: send2trash>=1.8.2 in /opt/conda/lib/python3.11/site-packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.8.2)
 Requirement already satisfied: terminado>=0.8.3 in /opt/conda/lib/python3.11/site-packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (0.17.1)
 Requirement already satisfied: websocket-client in /opt/conda/lib/python3.11/site-packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.6.4)
 Requirement already satisfied: babel>=2.10 in /opt/conda/lib/python3.11/site-packages (from jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (2.13.0)
 Requirement already satisfied: json5>=0.9.0 in /opt/conda/lib/python3.11/site-packages (from jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (0.9.14)
 Requirement already satisfied: jsonschema>=4.18.0 in /opt/conda/lib/python3.11/site-packages (from jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (4.19.1)
 Requirement already satisfied: requests>=2.31 in /opt/conda/lib/python3.11/site-packages (from jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (2.31.0)
 Requirement already satisfied: fastjsonschema in /opt/conda/lib/python3.11/site-packages (from nbformat>=5.7->nbconvert->jupyter) (2.18.1)
 Requirement already satisfied: wcwidth in /opt/conda/lib/python3.11/site-packages (from prompt-toolkit>=3.0.30->jupyter-console->jupyter) (0.2.8)
 Requirement already satisfied: soupsieve>1.2 in /opt/conda/lib/python3.11/site-packages (from beautifulsoup4->nbconvert->jupyter) (2.5)
 Requirement already satisfied: idna>=2.8 in /opt/conda/lib/python3.11/site-packages (from anyio>=3.1.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (3.4)
 Requirement already satisfied: sniffio>=1.1 in /opt/conda/lib/python3.11/site-packages (from anyio>=3.1.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.3.0)
 Requirement already satisfied: parso<0.9.0,>=0.8.3 in /opt/conda/lib/python3.11/site-packages (from jedi>=0.16->ipython>=7.23.1->ipykernel->jupyter) (0.8.3)
 Requirement already satisfied: attrs>=22.2.0 in /opt/conda/lib/python3.11/site-packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (23.1.0)
 Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /opt/conda/lib/python3.11/site-packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (2023.7.1)
 Requirement already satisfied: referencing>=0.28.4 in /opt/conda/lib/python3.11/site-packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (0.30.2)
 Requirement already satisfied: rpds-py>=0.7.1 in /opt/conda/lib/python3.11/site-packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (0.10.6)
 Requirement already satisfied: python-json-logger>=2.0.4 in

/opt/conda/lib/python3.11/site-packages (from jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (2.0.7)
 Requirement already satisfied: pyyaml>=5.3 in /opt/conda/lib/python3.11/site-packages (from jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (6.0.1)
 Requirement already satisfied: rfc3339-validator in /opt/conda/lib/python3.11/site-packages (from jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (0.1.4)
 Requirement already satisfied: rfc3986-validator>=0.1.1 in /opt/conda/lib/python3.11/site-packages (from jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (0.1.1)
 Requirement already satisfied: ptyprocess>=0.5 in /opt/conda/lib/python3.11/site-packages (from pexpect>4.3->ipython>=7.23.1->ipykernel->jupyter) (0.7.0)
 Requirement already satisfied: charset-normalizer<4,>=2 in /opt/conda/lib/python3.11/site-packages (from requests>=2.31->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (3.3.0)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/conda/lib/python3.11/site-packages (from requests>=2.31->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (2.0.7)
 Requirement already satisfied: certifi>=2017.4.17 in /opt/conda/lib/python3.11/site-packages (from requests>=2.31->jupyterlab-server<3,>=2.19.0->jupyterlab->jupyter) (2023.7.22)
 Requirement already satisfied: argon2-cffi-bindings in /opt/conda/lib/python3.11/site-packages (from argon2-cffi->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (21.2.0)
 Requirement already satisfied: executing>=1.2.0 in /opt/conda/lib/python3.11/site-packages (from stack-data->ipython>=7.23.1->ipykernel->jupyter) (1.2.0)
 Requirement already satisfied: asttokens>=2.1.0 in /opt/conda/lib/python3.11/site-packages (from stack-data->ipython>=7.23.1->ipykernel->jupyter) (2.4.0)
 Requirement already satisfied: pure-eval in /opt/conda/lib/python3.11/site-packages (from stack-data->ipython>=7.23.1->ipykernel->jupyter) (0.2.2)
 Requirement already satisfied: fqdn in /opt/conda/lib/python3.11/site-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.5.1)
 Requirement already satisfied: isoduration in /opt/conda/lib/python3.11/site-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (20.11.0)
 Requirement already satisfied: jsonpointer>1.13 in /opt/conda/lib/python3.11/site-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (2.4)
 Requirement already satisfied: uri-template in /opt/conda/lib/python3.11/site-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.3.0)
 Requirement already satisfied: webcolors>=1.11 in


```

/opt/conda/lib/python3.11/site-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (1.13)
Requirement already satisfied: cffi>=1.0.1 in /opt/conda/lib/python3.11/site-
packages (from argon2-cffi-bindings->argon2-cffi->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (1.16.0)
Requirement already satisfied: pycparser in /opt/conda/lib/python3.11/site-
packages (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (2.21)
Requirement already satisfied: arrow>=0.15.0 in /opt/conda/lib/python3.11/site-
packages (from isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.3.0)
Requirement already satisfied: types-python-dateutil>=2.8.10 in
/opt/conda/lib/python3.11/site-packages (from
arrow>=0.15.0->isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.6.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (2.8.19.14)

```

[]:

Una vez instalado Jupyter, lanza el servidor Jupyter con el siguiente comando (como usuario root):

```
jupyter notebook --ip=0.0.0.0 --no-browser --allow-root --NotebookApp.token='bigdata' --NotebookApp.password=''
```

[]:

```
!jupyter notebook --ip=0.0.0.0 --no-browser --allow-root --NotebookApp.
token='bigdata' --NotebookApp.password=''
```

```

[I 2026-03-12 15:32:06.332 ServerApp] Package notebook took 0.0000s to
import
[I 2026-03-12 15:32:06.374 ServerApp] Package jupyter_lsp took 0.0412s
to import
[W 2026-03-12 15:32:06.374 ServerApp] A
`_jupyter_server_extension_points` function was not found in jupyter_lsp.
Instead, a `_jupyter_server_extension_paths` function was found and will be used
for now. This function name will be deprecated in future releases of Jupyter
Server.
[I 2026-03-12 15:32:06.377 ServerApp] Package jupyter_server_mathjax
took 0.0026s to import
[I 2026-03-12 15:32:06.392 ServerApp] Package jupyter_server_terminals
took 0.0139s to import
[I 2026-03-12 15:32:06.393 ServerApp] Package jupyterlab took 0.0000s to
import
[I 2026-03-12 15:32:08.988 ServerApp] Package jupyterlab_git took
0.0623s to import
[I 2026-03-12 15:32:08.996 ServerApp] Package nbclassic took 0.0075s to
import
[W 2026-03-12 15:32:09.000 ServerApp] A
`_jupyter_server_extension_points` function was not found in nbclassic. Instead,
a `_jupyter_server_extension_paths` function was found and will be used for now.

```

This function name will be deprecated in future releases of Jupyter Server.

```
[I 2026-03-12 15:32:09.001 ServerApp] Package nbdtme took 0.0000s to
import
[I 2026-03-12 15:32:09.001 ServerApp] Package notebook_shim took 0.0000s
to import
[W 2026-03-12 15:32:09.002 ServerApp] A
`_jupyter_server_extension_points` function was not found in notebook_shim.
Instead, a `_jupyter_server_extension_paths` function was found and will be used
for now. This function name will be deprecated in future releases of Jupyter
Server.
[I 2026-03-12 15:32:09.003 ServerApp] jupyter_lsp | extension was
successfully linked.
[I 2026-03-12 15:32:09.007 ServerApp] jupyter_server_mathjax | extension
was successfully linked.
[I 2026-03-12 15:32:09.012 ServerApp] jupyter_server_terminals |
extension was successfully linked.
[W 2026-03-12 15:32:09.014 LabApp] 'token' has moved from NotebookApp to
ServerApp. This config will be passed to ServerApp. Be sure to update your
config before our next release.
[W 2026-03-12 15:32:09.014 LabApp] 'password' has moved from NotebookApp
to ServerApp. This config will be passed to ServerApp. Be sure to update your
config before our next release.
[W 2026-03-12 15:32:09.018 ServerApp] ServerApp.token config is
deprecated in 2.0. Use IdentityProvider.token.
[I 2026-03-12 15:32:09.018 ServerApp] jupyterlab | extension was
successfully linked.
[I 2026-03-12 15:32:09.018 ServerApp] jupyterlab_git | extension was
successfully linked.
[I 2026-03-12 15:32:09.022 ServerApp] nbclassic | extension was
successfully linked.
[I 2026-03-12 15:32:09.022 ServerApp] nbdtme | extension was
successfully linked.
[I 2026-03-12 15:32:09.028 ServerApp] notebook | extension was
successfully linked.
[I 2026-03-12 15:32:09.392 ServerApp] notebook_shim | extension was
successfully linked.
[I 2026-03-12 15:32:09.432 ServerApp] notebook_shim | extension was
successfully loaded.
[I 2026-03-12 15:32:09.435 ServerApp] jupyter_lsp | extension was
successfully loaded.
[I 2026-03-12 15:32:09.435 ServerApp] jupyter_server_mathjax | extension
was successfully loaded.
[I 2026-03-12 15:32:09.436 ServerApp] jupyter_server_terminals |
extension was successfully loaded.
[I 2026-03-12 15:32:09.440 LabApp] JupyterLab extension loaded from
/opt/conda/lib/python3.11/site-packages/jupyterlab
[I 2026-03-12 15:32:09.440 LabApp] JupyterLab application directory is
/opt/conda/share/jupyter/lab
```

```
[I 2026-03-12 15:32:09.441 LabApp] Extension Manager is 'pypi'.
[I 2026-03-12 15:32:09.443 ServerApp] jupyterlab | extension was
successfully loaded.
[I 2026-03-12 15:32:09.450 ServerApp] jupyterlab_git | extension was
successfully loaded.
```

```

  _ _ _ _ _
 | | | | _ _ _ _ | | _ _ _ _
 | | | | ' \ _ _ / _ _ | _ _ _
 \ _ _ / | _ _ \ _ _ , \ _ _ , \ _ _ \ _ _
   | _ |

```

Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions.

https://jupyter-notebook.readthedocs.io/en/latest/migrate_to_notebook7.html

Please note that updating to Notebook 7 might break some of your extensions.

```
[I 2026-03-12 15:32:09.458 ServerApp] nbclassic | extension was
successfully loaded.
[I 2026-03-12 15:32:09.635 ServerApp] nbdime | extension was
successfully loaded.
[I 2026-03-12 15:32:09.640 ServerApp] notebook | extension was
successfully loaded.
[I 2026-03-12 15:32:09.641 ServerApp] The port 8888 is already in use,
trying another port.
[I 2026-03-12 15:32:09.641 ServerApp] Serving notebooks from local
directory: /tmp
[I 2026-03-12 15:32:09.641 ServerApp] Jupyter Server 2.8.0 is running
at:
[I 2026-03-12 15:32:09.641 ServerApp]
http://f3b61ef67c45:8889/tree?token=...
[I 2026-03-12 15:32:09.642 ServerApp]
http://127.0.0.1:8889/tree?token=...
[I 2026-03-12 15:32:09.642 ServerApp] Use Control-C to stop this server
and shut down all kernels (twice to skip confirmation).
[I 2026-03-12 15:32:11.154 ServerApp] Skipped non-installed server(s):
bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-
langserver, jedi-language-server, julia-language-server, pyright, python-
language-server, python-lsp-server, r-languageserver, sql-language-server,
texlab, typescript-language-server, unified-language-server, vscode-css-
languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-
bin, yaml-language-server
```

Con esto, el servidor Jupyter quedará escuchando en el puerto 8888 del contenedor del NameNode, que a su vez tiene mapeado el puerto al exterior, así que se podrá acceder desde el navegador web o desde Visual Studio Code.

En Visual Studio Code, hay que seleccionar el kernel:

Después, “Select Another Kernel...” y en la ventana que aparece, seleccionar “Existing Jupyter Server”:

Con la URL `http://localhost:8888/?token=bigdata`.

A partir de ahí, todos los *Notebooks* locales se ejecutarán en el contenedor del NameNode, con acceso al clúster Hadoop.

`http://localhost:8888/?token=bigdata`